'ENGINEERS' QUEST TO FAST APPROXIMATE COMPUTATIONS'

PARTH KARNAWAT, SIDDHARTH NAIK, PRAFUL SHANKAR, SOUMYABRATA KUNDU

Abstract. We explore three algorithms that are considerably fast for the purpose they were conceived. First we explore the Karatsuba algorithm for fast multiplication and compare it to the more familiar grade-school approach. Next we tackle the Fast Fourier Transform that is fundamental to signal processing. Lastly, we turn to the Quake's Fast Inverse algorithm to study it's theoretical underpinnings and to analyse the infamous magic number 0x5F3759DF.

Contents

1. Karatsuba Algorithm 2
1.1. Naive Approach 2
1.2. Analysis and Algorithm 2
1.3. Experimental Comparison 4
2. Fast Fourier Transform 5
2.1. Decimation in Time 5
2.2. Decimation in Frequency 6
2.3. Butterfly Diagrams 7
3. Quake's Fast Inverse Square Root 9
3.1. Introduction 9
3.2. Notation and Background 9
3.3. The Magic Number 10
3.4. Exponent R_1 10
3.5. Mantissa R_2 13
3.6. Combining Errors 15

1. Karatsuba Algorithm

The Karatsuba algorithm provides a striking example of how the "Divide and Conquer" technique can achieve an asymptotic speedup over an ancient algorithm. Computer scientists often consider multiplication to be a constant time $O(1)$ operation, and this is a reasonable simplification for smaller numbers; but for larger numbers, the actual running times need to be factored in, which is $O(n^2)$. The point of the Karatsuba algorithm is to break large numbers down into smaller numbers so that any multiplications that occur happen on smaller numbers. Karatsuba can be used to multiply numbers in all base systems (base-10, base-2, etc.).

1.1. **Naive Approach.** A Naive Approach is to follow the process we study in school. One by one take all digits of second number and multiply it with all digits of first number. Finally add all multiplications. This algorithm takes $O(n^2)$ time. We present this visually here:

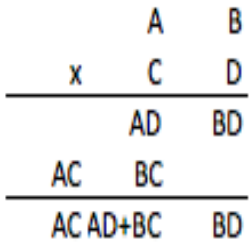


Figure 1. Rough Sketch of Naive Multiplication

This algorithm at face value seems like the best we could do. As recently as 1960, eminent mathematicians like A. Kolmogorov conjectured that we could not do better than the classical $\Omega(n^2)$ approach. But within a week, then 23 year old Anatoly Karatsuba discovered a faster "Divide and Conquer" approach that multiplies two n-digit numbers in $O(n^{\log_2 3})$ elementary steps, thus disproving the conjecture.

1.2. **Analysis and Algorithm.** We first consider what is happening in the naive method. Given two n-digit numbers A and B in base 10, we first split it into two parts:

$$A = \underbrace{(a_n \dots a_{\frac{n}{2}})}_{A_{high}} \times 10^{\frac{n}{2}} + \underbrace{(a_{\frac{n}{2}-1} \dots a_1)}_{A_{low}}$$

$$B = \underbrace{(b_n \dots b_{\frac{n}{2}})}_{B_{high}} \times 10^{\frac{n}{2}} + \underbrace{(b_{\frac{n}{2}-1} \dots b_1)}_{B_{low}}$$

We can now write the naive multiplication as:

$$A \times B = 10^n A_{high} B_{high} + 10^{\frac{n}{2}} (A_{high} B_{low} + A_{low} B_{high}) + A_{low} B_{low} \tag{1.1}$$

Equation (1.1) describes what is going on in the naive multiplication that we are familiar with. There are essentially four multiplications that are being computed in every iteration. Karatsuba's insight was precisely the fact that:

(1.2)
$$A_{high}B_{low} + A_{low}B_{high} = (A_{high} + A_{low}) \times (B_{high} + B_{low}) - A_{high}B_{high} + A_{low}B_{low}$$

This approach clearly reduces the number of multiplications required from four to three. And using a recursive approach we present a SAGEMath implementation of the Karatsuba Algorithm:

```
1  def karatsuba(a,b):
2
3      if abs(a) < 10  or abs(b) < 10:
4          return a*b
5
6      else:
7          size = max(len(str(a)),len(str(b)))
8          cut = size // 2
9
10         high_a = a // 10**(cut)
11         low_a = a % 10**(cut)
12         high_b = b // 10**(cut)
13         low_b = b % 10**(cut)
14
15         z2 = karatsuba(high_a,high_b)
16         z1 = karatsuba((low_a + high_a),(low_b+high_b))
17         z0 = karatsuba(low_a,low_b)
18
19         return (z2*10**(2*cut))+(\
20             (z1 - z2 - z0) * 10**(cut)) + (z0)
```

Analysis: This is a recursive algorithm: during execution, it calls smaller instances of itself. Let $M(n)$ denote the number of digit-multiplications (line 4) of our code, where $n = 2^k$ for some k. In lines 15, 16 and 17 the algorithm calls itself three times on $n/2$ digit integers; therefore

$$M(n) = 3M(\frac{n}{2}) \tag{1.3}$$

This equation is a simple recurrence which we may solve directly as follows. Applying equation (1.3) to $M(\frac{n}{2})$ we obtain $M(\frac{n}{2}) = 3M(\frac{n}{4})$; therefore $M(n) = 9M(\frac{n}{4})$. Continuing similarly we see that $M(n) = 27M(\frac{n}{8})$, and it follows by induction on i that for every i ($i \leq k$),

$$M(n) = 3^i M(\frac{n}{2^i}) \tag{1.4}$$

Setting $i = k$ we find that $M(n) = 3^k M(\frac{n}{2^k}) = 3^k M(1) = 3^k$. Notice that $k = \log_2 n$ (base 2 logarithm), therefore $\log_2 M(n) = k \log_2 3$ and hence

$$M(n) = 2^{\log_2 M(n)} = 2^{k \log_2 3} = n^{\log_2 3} \tag{1.5}$$

There is a trade-off for reducing the number of digit-multiplications from n^2 to $n^{\log_2 3}$. The cost is an increased number of additions (lines 16,19 and 20). Appearances are deceptive: actually, the procedure achieves similar savings in terms of total number of digit-operations (additions as well as multiplications). To see this, let $T(n)$ be the total number of digit-operations required by the algorithm. Then

$$T(n) = 3T(\frac{n}{2}) + O(n) \tag{1.6}$$

where the term $3T(\frac{n}{2})$ comes, from lines 15, 16 and 17; the additional $O(n)$ term is the number of digit-additions required to perform the additions and subtractions in lines 16, 19 and 20. We include bookkeeping costs in this term. It turns out that the additive $O(n)$ doesn't change the order of magnitude, and the result will still be:

$$T(n) = O(n^{\log_2 3}) \simeq O(n^{1.585}) \tag{1.7}$$

1.3. **Experimental Comparison.** In our attached code you can find an implementation of the Naive algorithm as well as the Karatsuba function. Using the time module we compared the time efficiencies of the two approaches and obtained the following results:

```
Prafuls-MacBook-Air:Project prafulshankar$ python -i karatsuba.py
a               b               naive           karatsuba       naive_time          karatsuba_time
431501          860891          371475327391    371475327391    4.839897E-04        2.789497E-05
625663          154252          96509769076     96509769076     4.098415E-04        2.408028E-05
970863          1940            1883474220      1883474220      2.911091E-04        2.098083E-05
716998          328181          235305120638    235305120638    4.041195E-04        2.312660E-05
173817          40723           7078349691      7078349691      4.441738E-04        2.694130E-05
365825          580904          212509205800    212509205800    3.941059E-04        2.980232E-05
295971          904463          267694818573    267694818573    4.038811E-04        3.004074E-05
760115          524048          398336745520    398336745520    3.941059E-04        2.598763E-05
205219          786352          161374371088    161374371088    5.021095E-04        2.884865E-05
495101          926632          458776429832    458776429832    5.488396E-04        1.320839E-04
379878          416797          158332010766    158332010766    4.251003E-04        3.600121E-05
451357          466368          210498461376    210498461376    4.408360E-04        2.908707E-05
699913          109483          76628574979     76628574979     4.000664E-04        3.004074E-05
490208          71369           34985654752     34985654752     3.380775E-04        1.883507E-05
474076          883200          418703923200    418703923200    3.919601E-04        2.503395E-05
456003          998402          455274307206    455274307206    5.509853E-04        1.907349E-05
79187           216286          17127039482     17127039482     3.590584E-04        2.193451E-05
314052          297849          93540074148     93540074148     7.081032E-04        4.887581E-05
68776           360198          24772977648     24772977648     4.999638E-04        3.194809E-05
651050          711923          463497469150    463497469150    4.150867E-04        1.597404E-05
>>>
```

FIGURE 2. Note the last two columns

Figure (2) should make it very clear that Karatsuba outperforms the Naive algorithm. There have since been many methods discovered that are even faster and could be explored by the reader.

2. Fast Fourier Transform

Discrete Fourier transforms (DFTs) are extremely useful because they reveal periodicities in input data as well as the relative strengths of any periodic components. There are however a few subtleties in the interpretation of discrete Fourier transforms. In general, the discrete Fourier transform of a real sequence of numbers will be a sequence of complex numbers of the same length.

Let $x_0, ..., x_{N?1}$ be complex numbers. The DFT is defined by the formula:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N} \qquad k = 0, \ldots, N-1 \tag{2.1}$$

where $e^{i2\pi/N}$ is a primitive N^{th} root of 1.

For a particular frequency k in $\{0, 1, \ldots, n-1\}$there are N terms in the summation. Each of those terms is the product of 2 complex numbers. So for each frequency we require N complex multiplication. For all N frequency we need N^2 complex multiplications. Time complexity is $O(N^2)$.
Fast Fourier transform is a way of implementation of DFT credited to Cooley and Tuckey to compute the DFT of a given sequence of real or complex numbers whose time complexity is $O(N \log N)$ which is the best known performance bound. It uses to the repetitive nature of the Nth roots of unity. There are two algorithms we will discuss. We shall restrict ourselves to radix-2 case ,i.e, N is a power of 2.

2.1. Decimation in Time.

A radix-2 decimation-in-time (DIT) FFT is the simplest and most common form of the Cooley?Tukey algorithm. Radix-2 DIT divides a DFT of size N into 2 interleaved DFTs (hence the name "radix-2") of size $\frac{N}{2}$ with each recursive stage. The discrete Fourier transform (DFT) is defined by the formula:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i}{N} nk} \tag{2.2}$$

The idea is to separate the odd and even index entries from the summation. To break it into to two summations of length N/2 each:

$$X_k = \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N}(2m)k} + \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N}(2m+1)k}$$

$$X_k = \underbrace{\sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} mk}}_{\text{DFT of even-indexed part of } x_n} + e^{-\frac{2\pi i}{N}k} \underbrace{\sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} mk}}_{\text{DFT of odd-indexed part of } x_n}$$

$$X_k = E_k + e^{-\frac{2\pi i}{N}k} O_k$$

Due to periodicity of complex numbers $X_{k+\frac{n}{2}}$ can also be obtained from E_k and O_k.

$$\begin{aligned}
X_{k+\frac{N}{2}} &= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} m(k+\frac{N}{2})} + e^{-\frac{2\pi i}{N}(k+\frac{N}{2})} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} m(k+\frac{N}{2})} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} mk} e^{-2\pi mi} + e^{-\frac{2\pi i}{N} k} e^{-\pi i} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} mk} e^{-2\pi mi} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} mk} - e^{-\frac{2\pi i}{N} k} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} mk} \\
&= E_k - e^{-\frac{2\pi i}{N} k} O_k
\end{aligned}$$

We can rewrite X_i as:

$$\begin{aligned}
X_k &= E_k + e^{-\frac{2\pi i}{N} k} O_k \\
X_{k+\frac{N}{2}} &= E_k - e^{-\frac{2\pi i}{N} k} O_k
\end{aligned}$$

This is the recursion formula we shall use.

2.2. **Decimation in Frequency.** In this method we shall group the N terms in two groups. The first $\frac{N}{2}$ terms in the first group and the rest $\frac{N}{2}$ terms in another.

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N} \qquad k = 0, \ldots, N-1 \tag{2.3}$$

Where N is a power of 2. We substitute $W_N = e^{\frac{-j2\pi}{N}}$. So X_k can be expressed as:

$$\begin{aligned}
X_k &= \sum_{n=0}^{N/2-1} x_n W_N^{kn} + \sum_{n=N/2}^{N-1} x_n W_N^{kn} \\
X_k &= \sum_{n=0}^{N/2-1} x_n W_N^{kn} + \sum_{n=0}^{N/2-1} x_{n+N/2} W_N^{kn} \\
X_k &= \sum_{n=0}^{N/2-1} x_n W_N^{kn} + W_N^{kN/2} \sum_{n=0}^{N/2-1} x_n W_N^{kn}
\end{aligned}$$

But:

$$W_N^{(N/2)k} = e^{2\pi k(N/2)/N} = e^{\pi k} = (-1)^k$$

Then:

$$X_k = \sum_{n=0}^{N/2-1} x_n W_N^{kn} + (-1)^k \sum_{n=0}^{N/2-1} x_{n+N/2} W_N^{kn}$$

If $k = 2m$ or an even number:

$$X_{2m} = \sum_{n=0}^{N/2-1} (x_n + x_{n+N/2} W_N^{2mn}$$
$$X_{2m} = \sum_{n=0}^{N/2-1} a_n W_{N/2}^{mn}$$

Noting that:

$$W_N^{2mn} = e^{-j2mn\times 2\pi/N} = e^{-j2\pi mn/(N/2)} = W_{N/2}^{mn}$$
$$a_n = x_n + x_{n+N/2}$$

Then X_2m is $N/2$- point DFT for a_n. If $k = 2m+1$ and using the same method:

$$X_{2m+1} = \sum_{n=0}^{N/2-1} (x_n - x_{n+N/2}) W_N^n W_N^{2mn}$$
$$X_{2m+1} = \sum_{n=0}^{N/2-1} (b_n W_N^{)} W_{N/2}^{mn}$$

Where $b_n = x_n - x_{n+N/2}$. So X_{2m+1} is $N/2$-point DFT for $b_n W_N^n$.

2.3. **Butterfly Diagrams.** These Algorithms can be represented as diagrams called butterfly diagrams, represented below. The diagram shows the algorithm for $N = 8$.

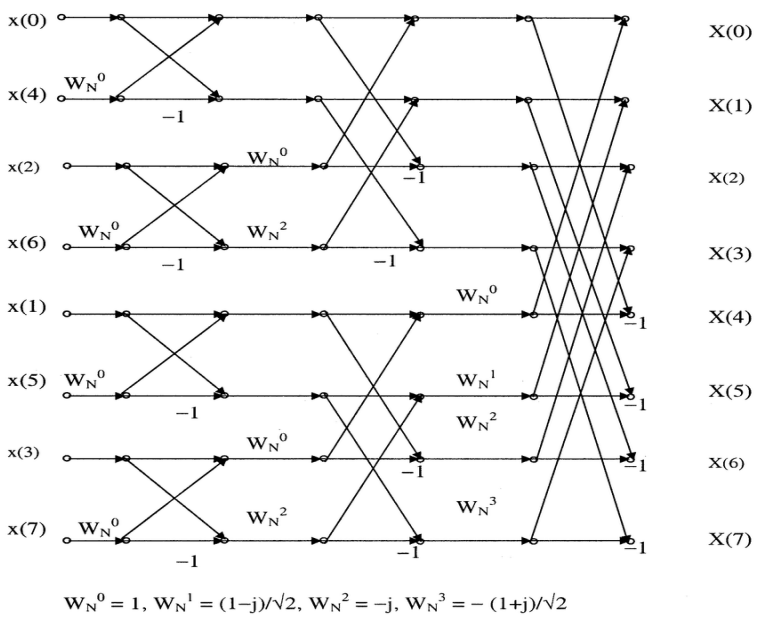


FIGURE 3. Butterfly Diagram for DIT

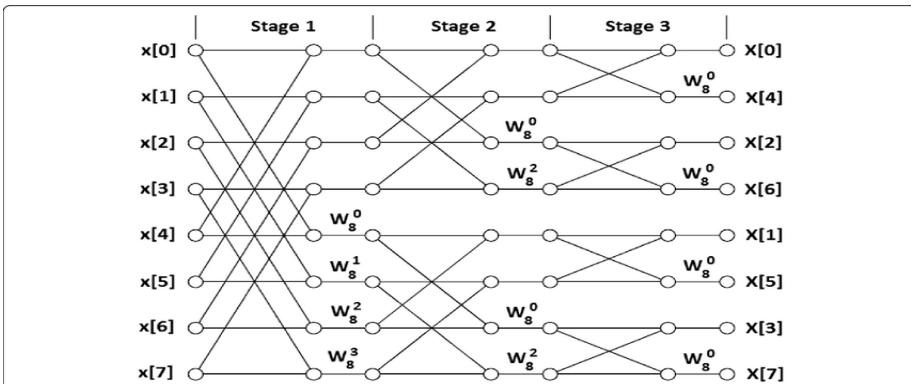


FIGURE 4. Butterfly Diagram for DIF

In the code submitted there are 3 functions:

- Main: It takes two inputs, a list of data(list type) and the algorithm to invoke(string). The possible choices for the string are 'dit' and 'dif'. Main checks whether the length of data is a power of two and then invokes the specified algorithm.
- fft_dit : For implementing the algorithm of decimation in time.
- fft_dif : For implementing the algorithm of decimation in time.

3. Quake's Fast Inverse Square Root

3.1. **Introduction.** Computing reciprocal square roots is integral to many modern digital processes, like in vector normalisation in video games. As we will see, a certain "magic number" is integral to the creation of this algorithm. This magic number, 0x5f3759df, has a contested history which adds to the mystery behind its origins. It is not known precisely how the exact value for the magic number was determined. In our SAGEMath code, this is what the implementation looks like:

```
1  def invSqrt(x,n = 1):
2      x = x.n(24)            #gets bits for floating value
3      if(x<=0):
4          return NaN
5      i = 0x5f375a86 - (toIEEE(x)>>1)  #gives initial guess
6      i = toFloat(i)
7      for j in range(n):
8          i = i*(1.5-i*i*x/2)            #Newton Step
9      return i
```

For reference, this is from our code from lines 20-28. The algorithm is fairly simple to understand because the crux of it lies in line 5 of our code. It computes the initial guess y_0 using only one subtraction from an arbitrary seeming number and a right-shift operator ">>". We presume that the motivation here was clearly to exploit the way a computer stores numbers because the shift creates a linear interpolation to the inverse square root.

Another interesting part is the magic number used in line 5. Some quick tests show that this works roughly 4 times faster than naive calculations, and the maximum relative error over all floating point numbers was 0.00175228, so the method seems very useful. This code appears in the source code for the game Quake 3 but also appears in the Crystal Space, The Titan Engine and the Fast Code's source codes although each seems to derive from Quake 3. After some investigation it was concluded that the original algorithm was devised by Greg Walsh at Ardent Computer in consultation with Cleve Moler, the creator of MATLAB.

3.2. **Notation and Background.** Floating point numbers are stored in a regular PC as 32 bit numbers:

s	E	M
$bit\ 31$	$30 \leftarrow bits \rightarrow 23$	$22 \leftarrow bits \rightarrow 0$

Figure 5. Breakdown of the Bits

s is a 1 bit sign(1 denotes negative), while E is an 8 bit exponent, and M is a 23 bit mantissa. The exponent is biased by 127 to accommodate positive and negative exponents, and the mantissa does not store the leading 1, so think of M as

a binary number with the decimal point to the left. For convenience we introduce some notation:

$$E : \text{Value stored in Exponent bits}$$
$$e : \text{True exponent without the bias } (e = E - 127)$$
$$M : \text{Value stored in Mantissa bits}$$
$$m : \frac{M}{2^{23}}$$

Here e is the true exponent and m is a value in the interval [0, 1). So given a floating point x, the represented value is:

$$x = (-1)^s(1+m)2^e \tag{3.1}$$

We will now explore the theory behind the derivation of the Magic Number(s).

3.3. **The Magic Number.** As input we have a floating point value $x > 0$, which has a corresponding $s - E - M$ breakdown when in its 32-bit IEEE representation as above. The desired value y is therefore:

$$y = \frac{1}{\sqrt{x}} = \frac{1}{\sqrt{1+m}} 2^{\frac{-e}{2}} \tag{3.2}$$

For the general case we take a magic constant R, and analyse $y_0 = R - (i >> 1)$, where the subtraction is as integers, and i is the bit representation of x. We can represent R in the following way:



FIGURE 6. Breakdown of the Bits

i.e, R_1 is the exponent and R_2 is the mantissa. When we shift i right one bit, we may shift an exponent bit into the mantissa field, so will analyse two cases.

3.4. **Exponent R_1.** We are working with the hypothesis that such an R exists, i.e., given an input x we can approximate it's inverse square root by:

$$\frac{1}{\sqrt{x}} \simeq R - (x >> 1) \tag{3.3}$$

Using our notations and the way floating points are related to their exponents and mantissa, we have:

$$
\begin{aligned}
x &= (1+m)2^e \\
\frac{1}{\sqrt{x}} &= \frac{1}{\sqrt{1+m)}} 2^{\frac{-e}{2}} \\
\frac{1}{\sqrt{x}} &= \frac{2}{\sqrt{1+m)}} 2^{\frac{-e}{2}-1}
\end{aligned}
$$

Intuitively, we can see that since $m = \frac{M}{2^{23}}$, the value in the last denominator is very close to 1. Substituting $e' = \frac{-e}{2} - 1$, and noting that $\frac{2}{\sqrt{1+m}} \simeq (1+m')$ for some $0 < m' < 1$, we have:

$$\frac{1}{\sqrt{x}} = (1+m')2^{e'}$$

We want to use the relation between E and e to derive the exponent E' that corresponds to e', the exponent of $\frac{1}{\sqrt{x}}$:

$$
\begin{aligned}
E' - 127 &= \frac{-(E-127)}{2} - 1 \\
E' &= \frac{(127-E)}{2} + 126
\end{aligned}
$$

$$E' = 189.5 - \frac{E}{2} \tag{3.4}$$

Comparing equations (3.3) and (3.4), we conclude that the exponent R_1 of the magic number R, has to be in the vicinity of 189.5. But since we know that R_1 is an integer, we approximate it to 190. For a more complete treatment of this derivation, refer to this link. Numbers with exponent 190 have a hexadecimal representation beginning with 0x5f. Thus we have derived a small part of the magic number. We now analyse the errors associated with this exponent and try to find R_2 which minimises the error given this $R1$. This divides into two cases, when E is even or odd. We show the approach when E is even to derive an error of ϵ_0 and a similar approach will give an error ϵ_1 for the case when E is odd.

Even Case: Since the exponent is even, no bits from it shift into the mantissa under $i >> 1$, so the new mantissa is $R_2 - \lfloor \frac{m}{2} \rfloor$ (as integers), assuming $R_2 \geq \lfloor \frac{m}{2} \rfloor$. The initial guess is then:

$$
\begin{aligned}
y_0 &= (1 + R_2 - \frac{m}{2})2^{(R_1-64-d)-127} \\
&= (1 + R_2 - \frac{m}{2})2^{R_1-191-d} \\
&= (2 + 2R_2 - m)2^{R_1-192-d}
\end{aligned}
$$

If $R_2 < \frac{m}{2}$, then upon subtracting $R - (i >> 1)$ as integers, the bits in the mantissa field will have to borrow one from the exponent field, thus dividing y_0 by

two. The bits in the mantissa field will then be $(1 + R_2) - \lfloor \frac{m}{2} \rfloor$, which are still in $[0, 1)$. Thus if $R_2 < \frac{m}{2}$:

$$\begin{aligned} y_0 &= (1 + (1 + R_2 - \frac{m}{2}))2^{(R_1 - 64 - d) - 127 - 1} \\ &= (2 + R_2 - \frac{m}{2})2^{R_1 - 192 - d} \end{aligned}$$

where we write the exponent the same as in the non-carrying case. If we define:

$$\beta_m^{R_2} = \begin{cases} 2R_2 - m, & \text{if } R_2 \geq \frac{m}{2} \\ R_2 - \frac{m}{2}, & \text{if } R_2 < \frac{m}{2} \end{cases} \tag{3.5}$$

Then we can combine the equations into one:

$$y_0 = (2 + \beta_m^{R_2})2^{(R_1 - 192 - d)} \tag{3.6}$$

Note that $\beta_m^{R_2})2$ is continuous, and differentiable except along $R = \frac{m}{2}$. Substituting $e = 2d + 1$, the value $y = \frac{1}{\sqrt{x}}$ we are trying to approximate is:

$$\frac{1}{\sqrt{1+m}}2^{-\frac{e}{2}} = \frac{1}{\sqrt{1+m}}2^{-d-\frac{1}{2}} = \frac{1}{\sqrt{2+2m}}2^{-d}$$

The relative error $\left| \frac{y - y_0}{y} \right|$, which is how we will measure the quality of the method, is:

$$\left| \frac{\frac{1}{\sqrt{2+2m}}2^{-d} - (2 + \beta_m^{R_2})2^{R_1 - 192 - d}}{\frac{1}{\sqrt{2+2m}}2^{-d}} \right|$$

which simplifies to $|\epsilon_0(m, R)|$,

$$\epsilon_0(m, R) = 1 - \sqrt{2}\sqrt{1+m}(2 + \beta_m^{R_2})2^{R_1 - 192} \tag{3.7}$$

A similar approach works for when E is odd. For a more thorough treatment refer to the paper by Chris Lomont linked here. Here we present the results of this approach:

$$\epsilon_1(m, R) = 1 - \sqrt{1+m}(2 + \gamma_m^{R_2})2^{R_1 - 192} \tag{3.8}$$

where $\gamma_m^{R_2}$ is defined as:

$$\gamma_m^{R_2} = \begin{cases} 4R_2 - 2m, & \text{if } 2R_2 \geq m + 1 \\ 1 + 2R_2 - m, & \text{if } 2R_2 < m + 1 \end{cases} \tag{3.9}$$

We already have a good guess for the value of R_1 as 190, we substitute it in equations (3.8) and (3.7) to obtain:

$$\epsilon_0(m, R_2) = 1 - \sqrt{2}\sqrt{1+m}\frac{(2+\beta_m^{R_2})}{4}$$
$$\epsilon_1(m, R_2) = 1 - \sqrt{1+m}\frac{(2+\gamma_m^{R_2})}{4}$$

In our code, we have plotted error $max(|\epsilon_0|, |\epsilon_1|)$ vs. (m, R_2) to present where the magic number would tentatively lie. The following figures present our output:

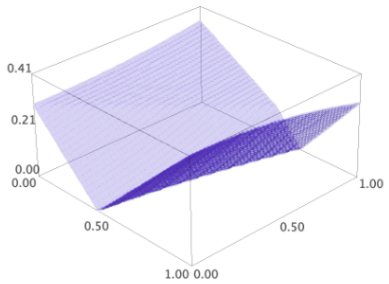


FIGURE 7. $max(|\epsilon_0|, |\epsilon_1|)$ vs. (m, R_2) over [0,1] x [0,1]

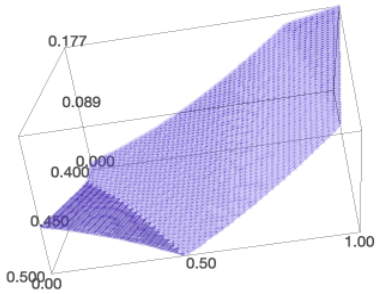


FIGURE 8. $max(|\epsilon_0|, |\epsilon_1|)$ vs. (m, R_2) over [0.35,0.45] x [0,1]

3.5. **Mantissa R_2.** Now that we have our R_1 and two error functions for given values of m and R_2, we turn out attention to optimising the Mantissa. We want a value of R_2 in [0, 1) that minimises $max_{m\in[0,1)}\{|\epsilon_0(m, R_2)|, |\epsilon_0(m, R_2)|\}$. Fix a value of R_2, and we will find the value of m giving the maximal error. Since the error values ϵ_0 and ϵ_1 are continuous and piecewise differentiable, this m will occur at an endpoint of a piece or at a critical point on a piece. This gives rise to nine different functions associated with each case. The nine functions are as follows:

3.5.1. *Maximising* $|\epsilon_0|$. Start with the endpoints for $\epsilon_0 : m = 0, m = 1$, and $R_2 = \frac{m}{2}$ (where $\beta_m^{R_2}$ is not differentiable). When $m = 0, \beta_m^{R_2} = 2R_2$. Let

$$f_1(R_2) = \epsilon_0(0, R_2) = 1 - \frac{\sqrt{2}\sqrt{1+0}(2+2R_2)}{4} = 1 - \frac{1+R_2}{\sqrt{2}}$$

Similarly, when $R_2 = \frac{m}{2}, \beta_m^{R_2} = 0$:

$$f_2(R_2) = \epsilon_0(2R_2, R_2) = 1 - \frac{\sqrt{2}\sqrt{1+2R_2}(2+0)}{4} = 1 - \sqrt{\frac{1+2R_2}{2}}$$

When $m = 1$, we need to consider the two cases for $\beta_m^{R_2}$, giving:

$$f_3(R_2) = \epsilon_0(1, R_2) = \begin{cases} \frac{1}{2}(1-2R_2) & \text{if } R_2 \geq \frac{1}{2} \\ \frac{1}{4}(1-2R_2), & \text{if } 2R_2 < \frac{1}{2} \end{cases}$$

Checking the critical points takes two cases. Assuming $R_2 \geq \frac{m}{2}$, solving $\frac{\partial \epsilon_0}{\partial m} = 0$ gives the critical point $m = \frac{2}{3}R_2$, giving:

$$f_4(R_2) = \epsilon_0(\frac{2R_2}{3}, R_2) = 1 - \frac{(1+\frac{2}{3}R_2)^{\frac{3}{2}}}{\sqrt{2}}$$

When $R - 2 < \frac{m}{2}$, the critical point is $m = \frac{2}{3}(1+R_2)$. This can only happen if R_2 is in [0, 1/2). so define:

$$f_5(R_2) = \begin{cases} 0 & \text{if } R_2 \geq \frac{1}{2} \\ \epsilon_0(\frac{2}{3}(1+R_2), R_2) = 1 - \frac{(5+2R_2)^{\frac{3}{2}}}{6\sqrt{6}} & \text{if } R_2 < \frac{1}{2} \end{cases}$$

3.5.2. *Maximising* $|\epsilon_1|$. . Similar analysis on ϵ_1 yields:

$$f_6(R_2) = \epsilon_1(0, R_2) = \begin{cases} \frac{1}{2} & \text{if } R_2 \geq \frac{1}{2} \\ \frac{1}{4}(1-2R_2) & \text{if } R_2 < \frac{1}{2} \end{cases}$$

$$f_7(R_2) = \epsilon_1(1, R_2) = \begin{cases} 1 - \sqrt{2}R_2 & \text{if } R_2 \geq 1 \\ 1 - \frac{(1+R_2)}{\sqrt{2}} & \text{if } R_2 < 1 \end{cases}$$

At the point $R = \frac{m+1}{2}, \gamma_m^{R_2} = 2$, but this can only occur when $R_2 \geq \frac{1}{2}$, so we get

$$f_8(R_2) = \begin{cases} \epsilon_1(2R_2 - 1, R_2) = 1 - \sqrt{2R_2} & \text{if } R_2 \geq \frac{1}{2} \\ 0 & \text{if } R_2 < \frac{1}{2} \end{cases}$$

The critical points again need two cases. If $R_2 \geq \frac{m}{2} + \frac{1}{2}$ the critical point is $m = \frac{2R-2-1}{3}$. This only occurs if $R_2 \geq \frac{1}{2}$. Similarly for $R_2 < \frac{m+1}{2}$ we obtain the critical point $m = \frac{2R_2+1}{3}$, which require $R_2 < \frac{1}{2}$. So we define both at once with:

$$f_9(R_2) = \begin{cases} \epsilon_1(\frac{2R_2-1}{3}, R_2) = 1 - (\frac{2}{3}(1+R_2))^{\frac{3}{2}} & \text{if } R_2 \geq \frac{1}{2} \\ \epsilon_1(\frac{2R_2+1}{3}, R_2) = 1 - (\sqrt{2}(\frac{1+R_2}{3}))^{\frac{3}{2}} & \text{if } R_2 < \frac{1}{2} \end{cases}$$

Note all f_i are continuous except f_9.

3.6. **Combining Errors.** So, for a given R_2, the above 9 functions give 9 values, and the max of the absolute values is the max error for this R_2. So we now vary $R_2 \in [0, 1)$ to find the best one. (The code for these plots and functions is included in the file attached with the submission):

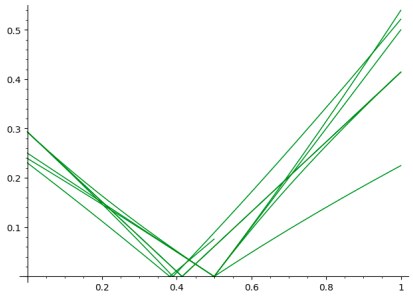


FIGURE 9. Errors generated by f_i's for given R_2

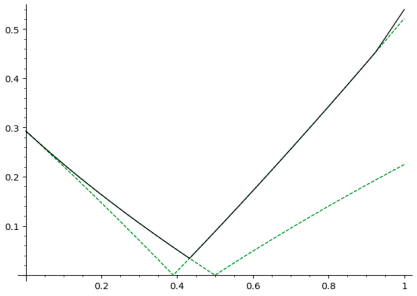


FIGURE 10. Isolating the Maximum functions

Figure 9 is a SAGE-Math plot of the error functions. The max of these is lowest near $R_2 = 0.45$; As you will see in our code, using bisection method we obtain a value of R_2 which minimises $max_i|f_i|$, returning:

$$r_0 = 0.432744889959443 \tag{3.10}$$

where this many digits are retained since this constant can be applied to any bit-length of floating point number. Note r_0 is very close to the original value in the code which is approximately 0.432430148. r_0 corresponds to the integer $R_2 = \lfloor 2^{23} r_0 + 0.5 \rfloor = 0$x37642f . Attaching $R_1 = 0$x5f to this we get $R = 0$x5f37642f.